LAPORAN PRAKTIKUM

Struktur Data

MODUL 5

SINGLY LINKED LIST (BAGIAN KEDUA)



Disusun oleh:

Farel Tri Julian

109082500163

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 5 : https://github.com/FarelTriJulian/Struktur-Data/tree/main/Modul5

1. Sll.h

Source Code

```
#ifndef SLL_H
#define SLL_H

typedef int infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next;
};

struct List {
    address First;
};

void createList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void insertFirst(List &L, address P);
void printInfo(List L);

// Fungsi tambahan soal nomor 3 dan 4
address findElm(List L, infotype x);
int totalInfo(List L);

#endif
```

Deskripsi program

Program sll.h ini isinya struktur dasar Single Linked List bertipe integer, lengkap sama deklarasi fungsi-fungsinya. Selain fungsi standar kayak buat list baru, alokasi memori, nambah data di awal (insertFirst), dan nampilkan isi list, ada juga dua fungsi tambahan buat nyari elemen (findElm) sama ngitung total nilai semua data di dalam list (totalInfo).

2. **Sll.cpp**

Source Code

```
#include "sll.h"
#include <iostream>
using namespace std;

void createList(List &L) {
    L.First = nullptr;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = nullptr;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = nullptr;
}

void insertFirst(List &L, address P) {
    P->next = L.First;
    L.First = P;
}

void printInfo(List L) {
    address P = L.First;

    while (P != nullptr) {
        cout << P->info;

        if (P->next != nullptr) {
            cout << " ";
        }

        P = P->next;
    }

    cout << endl;
}

address findElm(List L, infotype x) {
    address P = L.First;

    while (P != nullptr) {
```

```
            if (P->info == x) {
                return P;
            }

            P = P->next;
        }

        return nullptr;
    }

    int totalInfo(List L) {
        address P = L.First;
        int total = 0;

        while (P != nullptr) {
            total += P->info;
            P = P->next;
        }

        return total;
    }
```

Deskripsi program

Program sll.cpp ini adalah implementasi logika dari fungsi-fungsi di sll.h. Di sini diatur cara bikin list kosong (createList), alokasi dan hapus memori node (alokasi & dealokasi), nambah data ke urutan paling depan (insertFirst), serta nampilin isinya ke layar (printInfo). Ada juga fungsi traversal buat nyari posisi elemen sesuai nilainya (findElm) dan ngitung total penjumlahan seluruh data di dalam list (totalInfo).

3. **main.cpp**

Source Code

```
#include <iostream>
#include "sll.h"
using namespace std;

int main() {
    List L;
    address P1, P2, P3, P4, P5;

    P1 = P2 = P3 = P4 = P5 = nullptr;

    createList(L);

    P1 = alokasi(2);
```

```
    insertFirst(L, P1);

    P2 = alokasi(0);
    insertFirst(L, P2);

    P3 = alokasi(8);
    insertFirst(L, P3);

    P4 = alokasi(12);
    insertFirst(L, P4);

    P5 = alokasi(9);
    insertFirst(L, P5);

    printInfo(L);

    address hasil = findElm(L, 8);

    if (hasil != nullptr) {
        cout << "8 ditemukan dalam list" << endl;
    } else {
        cout << "8 tidak ditemukan dalam list" << endl;
    }

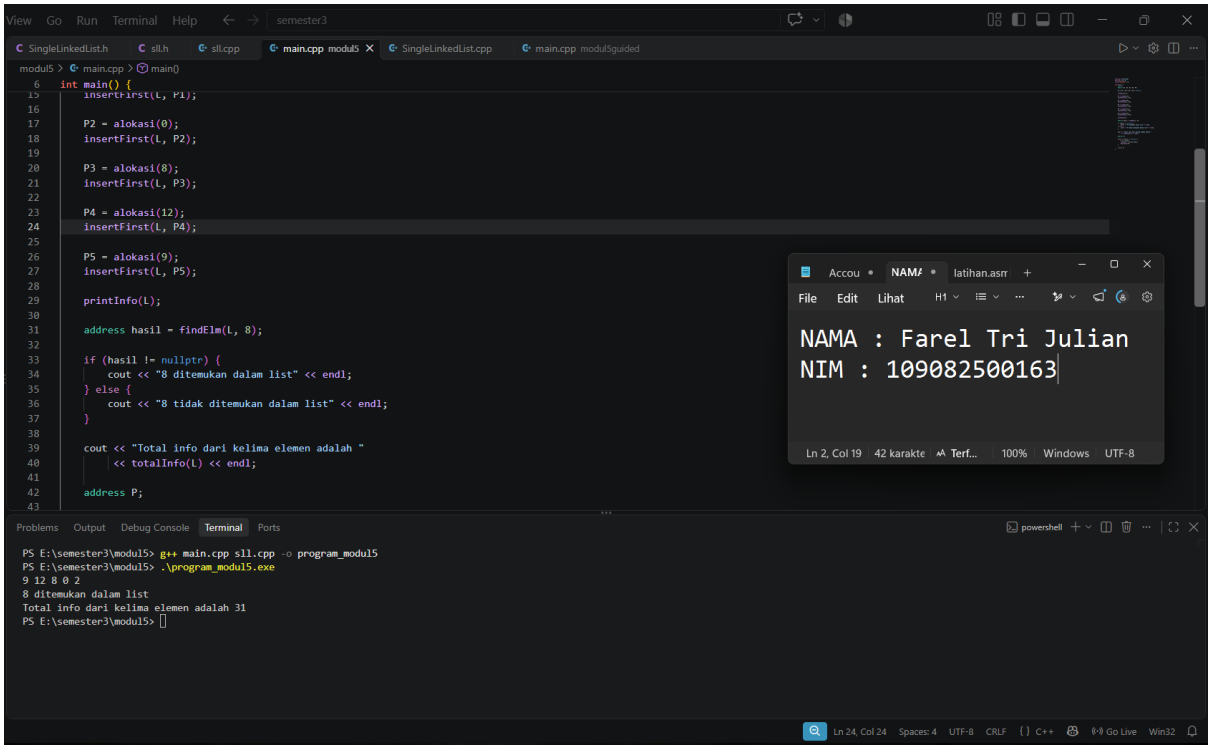
    cout << "Total info dari kelima elemen adalah "
         << totalInfo(L) << endl;

    address P;

    while (L.First != nullptr) {
        P = L.First;
        L.First = L.First->next;
        dealokasi(P);
    }

    return 0;
}
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah program utama buat nguji seluruh fungsi Single Linked List yang udah dibuat. Di sini program bikin list kosong, lalu ngisi lima node berturut-turut dengan angka 2, 0, 8, 12, dan 9 lewat insertFirst, sehingga urutan yang tampil jadi terbalik (9 12 8 0 2). Setelah nampilin isi list pakai printInfo, program bakal nyari keberadaan angka 8 pakai findElm, ngitung total penjumlahan seluruh elemen pakai totalInfo, dan ditutup sama proses *deallocation* buat ngehapus semua node dari memori sampai list-nya kosong lagi.

4. **SingleLinkedList.h (guided)**

Source Code

```
#ifndef SLL_H
#define SLL_H

struct ElmList {
    int info;
    ElmList* next;
};

struct List {
    ElmList* First;
};

// Fungsi dasar
void CreateList(List &L);
ElmList* alokasi(int nilai); //alokasi untuk me
void dealokasi(ElmList* &P);
void printInfo(List L);

// Searching
ElmList* findElm(List L, int nilai);
// Insert
void insertFirst(List &L, int nilai);
void insertLast(List &L, int nilai);
void insertAfter(List &L, int nilaiSebelum, int nilaiBaru);

// Delete
void delFirst(List &L);
void delLast(List &L);
void delAfter(List &L, int nilaiSebelum);
void delP(List &L, int nilai);

// Operasi tambahan
void update(List &L, int nilaiLama, int nilaiBaru);
int nbList(List L);
void delAll(List &L);

#endif
```

Deskripsi program

Program sll.h ini isinya struktur Single Linked List yang lebih lengkap dengan tipe data *integer* langsung. Selain fungsi dasar kayak alokasi dan cetak list, di sini udah disiapin operasi CRUD yang komplit: penambahan data di awal, akhir, atau setelah node tertentu (insertFirst, insertLast, insertAfter), berbagai cara penghapusan (delFirst, delLast, delAfter, delP), pencarian (findElm), sampai ubah nilai node (update). Ditambah lagi ada

fungsi utilitas buat ngitung jumlah total elemen (nbList) dan ngehapus semua isi list sekaligus (delAll).

5. **SingleLinkedList.cpp (guided)**

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

// Membuat list kosong
void CreateList(List &L) {
    L.First = nullptr;
}

// Membuat node baru
ElmList* alokasi(int nilai) {
    ElmList* P = new ElmList;

    P->info = nilai;
    P->next = nullptr;

    return P;
}

// Membebaskan memori node
void dealokasi(ElmList* &P) {
    delete P;
    P = nullptr;
}

// Menampilkan semua isi list
void printInfo(List L) {
    ElmList* P = L.First;

    while (P != nullptr) {
        cout << P->info << " ";
        P = P->next;
    }

    cout << endl;
}

// Mencari node berdasarkan nilai
ElmList* findElm(List L, int nilai) {
    ElmList* P = L.First;

    while (P != nullptr) {
```

```
            if (P->info == nilai) {
                return P;
            }

            P = P->next;
        }

        return nullptr;
    }

    // Menambahkan node di awal
    void insertFirst(List &L, int nilai) {
        ElmList* P = alokasi(nilai);

        P->next = L.First;
        L.First = P;
    }

    // Menambahkan node di akhir
    void insertLast(List &L, int nilai) {
        ElmList* P = alokasi(nilai);

        if (L.First == nullptr) {
            L.First = P;
        } else {
            ElmList* Q = L.First;

            while (Q->next != nullptr) {
                Q = Q->next;
            }

            Q->next = P;
        }
    }

    // Menambahkan node setelah nilai tertentu
    void insertAfter(List &L, int nilaiSebelum, int nilaiBaru) {
        ElmList* Q = findElm(L, nilaiSebelum);

        if (Q == nullptr) {
            cout << "Nilai sebelumnya tidak ditemukan." << endl;
        } else {
            ElmList* P = alokasi(nilaiBaru);

            P->next = Q->next;
            Q->next = P;
        }
    }
```

```
// Menghapus node pertama
void delFirst(List &L) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else {
        ElmList* P = L.First;

        L.First = L.First->next;
        dealokasi(P);
    }
}

// Menghapus node terakhir
void delLast(List &L) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else if (L.First->next == nullptr) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next->next != nullptr) {
            P = P->next;
        }

        ElmList* Q = P->next;
        P->next = nullptr;
        dealokasi(Q);
    }
}

// Menghapus node setelah nilai tertentu
void delAfter(List &L, int nilaiSebelum) {
    ElmList* P = findElm(L, nilaiSebelum);

    if (P == nullptr) {
        cout << "Nilai sebelumnya tidak ditemukan." << endl;
    } else if (P->next == nullptr) {
        cout << "Tidak ada node setelah nilai tersebut." <<
endl;
    } else {
        ElmList* Q = P->next;

        P->next = Q->next;
        dealokasi(Q);
    }
}
```

```
// Menghapus node berdasarkan nilai
void delP(List &L, int nilai) {
    if (L.First == nullptr) {
        cout << "List kosong." << endl;
    } else if (L.First->info == nilai) {
        delFirst(L);
    } else {
        ElmList* P = L.First;

        while (P->next != nullptr &&
               P->next->info != nilai) {
            P = P->next;
        }

        if (P->next == nullptr) {
            cout << "Nilai tidak ditemukan." << endl;
        } else {
            ElmList* Q = P->next;

            P->next = Q->next;
            dealokasi(Q);
        }
    }
}

// Mengubah nilai data
void update(List &L, int nilaiLama, int nilaiBaru) {
    ElmList* P = findElm(L, nilaiLama);

    if (P == nullptr) {
        cout << "Nilai tidak ditemukan." << endl;
    } else {
        P->info = nilaiBaru;
    }
}

// Menghitung jumlah node
int nbList(List L) {
    int jumlah = 0;
    ElmList* P = L.First;

    while (P != nullptr) {
        jumlah++;
        P = P->next;
    }

    return jumlah;
```

```
}

// Menghapus semua node
void delAll(List &L) {
    while (L.First != nullptr) {
        delFirst(L);
    }
}
```

Deskripsi program

Program SingleLinkedList.cpp ini berisi implementasi lengkap dari seluruh fungsi operasi Single Linked List. Di dalamnya diatur pembuatan dan manajemen memori list (CreateList, alokasi, dealokasi), penambahan node di berbagai posisi (insertFirst, insertLast, insertAfter), penghapusan data secara spesifik maupun menyeluruh (delFirst, delLast, delAfter, delP, delAll), serta fungsi utilitas untuk menampilkan isi list (printInfo), pencarian (findElm), pengubahan nilai (update), dan penghitungan total elemen (nbList).

6. Main.cpp (guided)

Source Code

```
#include "SingleLinkedList.h"
#include <iostream>
using namespace std;

int main() {
    List L;
    ElmList* hasil;
    ElmList* P;
    int total = 0;

    CreateList(L);

    // 1. Insert First
    insertFirst(L, 2);
    insertFirst(L, 0);
    insertFirst(L, 8);
    insertFirst(L, 12);
    insertFirst(L, 9);

    cout << "List awal: ";
    printInfo(L);

    // 2. Searching
    hasil = findElm(L, 8);

    if (hasil != nullptr) {
```

```
        cout << "Angka 8 ditemukan." << endl;
    } else {
        cout << "Angka 8 tidak ditemukan." << endl;
    }

    // 3. Menghitung total nilai
    P = L.First;

    while (P != nullptr) {
        total = total + P->info;
        P = P->next;
    }

    cout << "Total nilai: " << total << endl;

    // 4. Insert Last
    insertLast(L, 20);
    cout << "Setelah insertLast(20): ";
    printInfo(L);

    // 5. Insert After
    insertAfter(L, 12, 99);
    cout << "Setelah insertAfter(12, 99): ";
    printInfo(L);

    // 6. Update
    update(L, 99, 100);
    cout << "Setelah update(99, 100): ";
    printInfo(L);

    // 7. Delete After
    delAfter(L, 12);
    cout << "Setelah delAfter(12): ";
    printInfo(L);

    // 8. Delete berdasarkan nilai
    delP(L, 8);
    cout << "Setelah delP(8): ";
    printInfo(L);

    // 9. Delete First
    delFirst(L);
    cout << "Setelah delFirst: ";
    printInfo(L);

    // 10. Delete Last
    delLast(L);
    cout << "Setelah delLast: ";
```

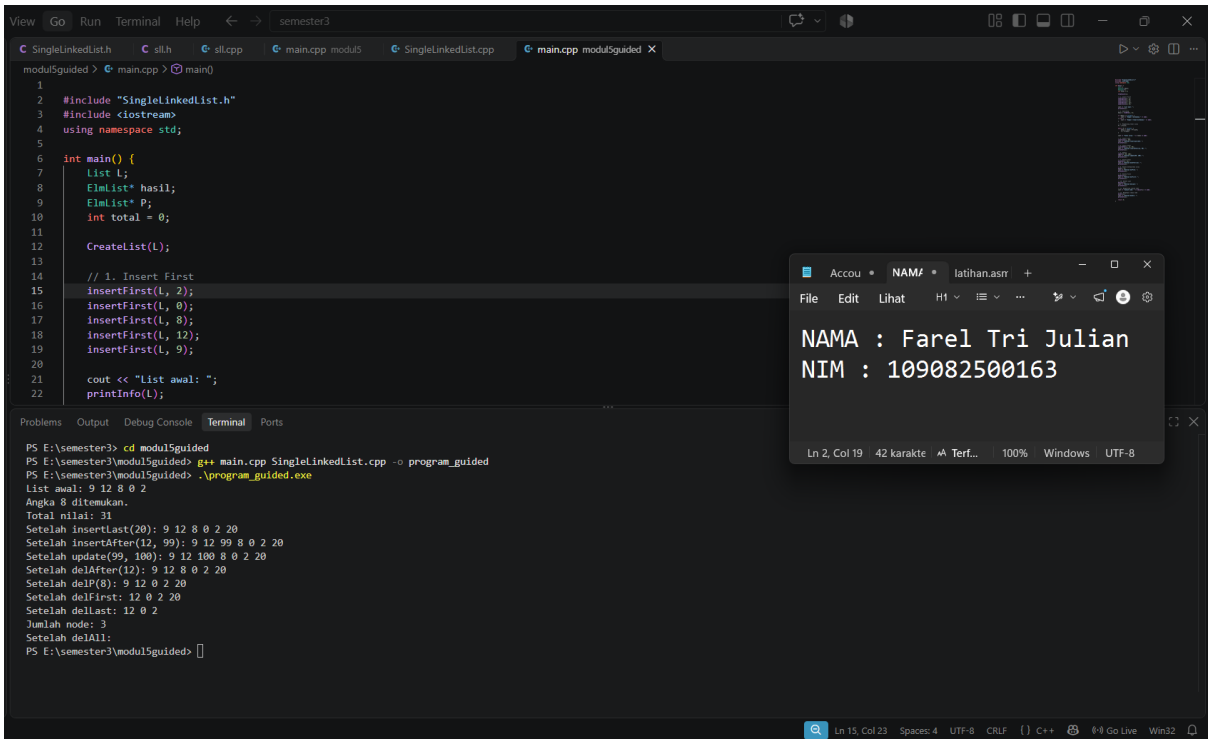
```
        printInfo(L);

        // 11. Menghitung jumlah node
        cout << "Jumlah node: " << nbList(L) << endl;

        // 12. Menghapus semua node
        delAll(L);
        cout << "Setelah delAll: ";
        printInfo(L);

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah program pengujian utama untuk mendemonstrasikan seluruh fitur Single Linked List dari SingleLinkedList.h. Program dimulai dengan mengisi data di awal (insertFirst), mencari angka 8, lalu menghitung total nilai seluruh elemen secara manual memakai perulangan while. Setelah itu, program menguji berbagai operasi penambahan, pengubahan, dan penghapusan data secara berurutan—seperti insertLast, insertAfter, update, delAfter, delP, delFirst, dan delLast. Di akhir alur, program menampilkan jumlah node tersisa dengan nbList dan mengosongkan seluruh isi list menggunakan delAll.